MC458 - Projeto e Análise de Algoritmos I

Prof. Dr. Hilder Vitor Lima Pereira

Instituto de computação

UNICAMP

31 de março de 2026

1 Comentários sobre o teste

2 Heapsort
- Motivação
- Representação

3 Perguntas ou comentários?

Comentários sobre o teste

- Invariantes incorretos: faltando variáveis ou com propriedades que não garantem a corretude

- Invariantes incorretos: faltando variáveis ou com propriedades que não garantem a corretude
- Código errado: soma começa em k=1; soma termina em n+1

- Invariantes incorretos: faltando variáveis ou com propriedades que não garantem a corretude
- Código errado: soma começa em k=1; soma termina em n+1
- Complexidade temporal $\Theta(n^2)$

- Invariantes incorretos: faltando variáveis ou com propriedades que não garantem a corretude
- Código errado: soma começa em k=1; soma termina em n+1
- Complexidade temporal $\Theta(n^2)$
  - Deixou redação mais longa pois teve invariantes para exponenciação e fatorial

- Invariantes incorretos: faltando variáveis ou com propriedades que não garantem a corretude
- Código errado: soma começa em k=1; soma termina em n+1
- Complexidade temporal $\Theta(n^2)$
  - Deixou redação mais longa pois teve invariantes para exponenciação e fatorial
- Não argumenta que laço termina com $k = n$ (ou $k = n + 1$) e isso implica corretude

- Invariantes incorretos: faltando variáveis ou com propriedades que não garantem a corretude
- Código errado: soma começa em k=1; soma termina em n+1
- Complexidade temporal $\Theta(n^2)$
  - Deixou redação mais longa pois teve invariantes para exponenciação e fatorial
- Não argumenta que laço termina com $k = n$ (ou $k = n + 1$) e isso implica corretude
- Laço "i = 1 to n": significa ¡= n? ¡ n?

Heapsort

Heapsort:

Motivação

Selection sort

```
SelectionSort(v, n)
Input: v ∈ ℕ^n
for 1 ≤ i < n do
    i_max ← FindMaximum(v, 0, n − i)
    v[i_max] ↔ v[n − i]
```

- Cada iteração acha o máximo do subvetor e o coloca na posição certa
- Invariante: o subvetor $v[n-i, ..., n-1]$ está ordenado

Selection sort

```
SelectionSort(v, n)
Input: v ∈ ℕ^n
for 1 ≤ i < n do
    i_max ← FindMaximum(v, 0, n − i)
    v[i_max] ↔ v[n − i]
```

- Cada iteração acha o máximo do subvetor e o coloca na posição certa
- Invariante: o subvetor $v[n-i, ..., n-1]$ está ordenado
- Complexidade:
  - suponha que $\texttt{FindMaximum}(v, 0, n-i)$ leva tempo $O(t(i))$
  - então o tempo total é

$$T(n) = \sum_{i=1}^{n-1} O(t(i)) \leq \sum_{i=1}^{n-1} O(t(n)) = O(n \cdot t(n))$$

Selection sort

```
SelectionSort(v, n)
Input: v ∈ ℕ^n
for 1 ≤ i < n do
    i_max ← FindMaximum(v, 0, n − i)
    v[i_max] ↔ v[n − i]
```

- Cada iteração acha o máximo do subvetor e o coloca na posição certa
- Invariante: o subvetor $v[n-i, ..., n-1]$ está ordenado
- Complexidade:
  - suponha que $\texttt{FindMaximum}(v, 0, n-i)$ leva tempo $O(t(i))$
  - então o tempo total é

$$T(n) = \sum_{i=1}^{n-1} O(t(i)) \leq \sum_{i=1}^{n-1} O(t(n)) = O(n \cdot t(n))$$

  - normalmente, temos $t(n) = \Theta(n)$ e concluímos que $T(n) \in O(n^2)$

Selection sort

```
SelectionSort(v, n)
Input: v ∈ ℕ^n
for 1 ≤ i < n do
    i_max ← FindMaximum(v, 0, n − i)
    v[i_max] ↔ v[n − i]
```

- Cada iteração acha o máximo do subvetor e o coloca na posição certa
- Invariante: o subvetor $v[n-i, ..., n-1]$ está ordenado
- Complexidade:
  - suponha que $\texttt{FindMaximum}(v, 0, n-i)$ leva tempo $O(t(i))$
  - então o tempo total é

$$T(n) = \sum_{i=1}^{n-1} O(t(i)) \leq \sum_{i=1}^{n-1} O(t(n)) = O(n \cdot t(n))$$

  - normalmente, temos $t(n) = \Theta(n)$ e concluímos que $T(n) \in O(n^2)$
  - Mas e se conseguíssemos executar $\texttt{FindMaximum}(v, 0, n-i)$ mais rapidamente?

O algoritmo HeapSort

Vamos estudar a fila de prioridade

- é uma estrutura de dados também chamada de max-heap
- implementa `FindMaximum` com tempo $O(\log n)$
- usando um heap, podemos ordenar em $O(n \log n)$
- esse algoritmo de ordenação é chamado de heapsort

Heapsort:

Representação

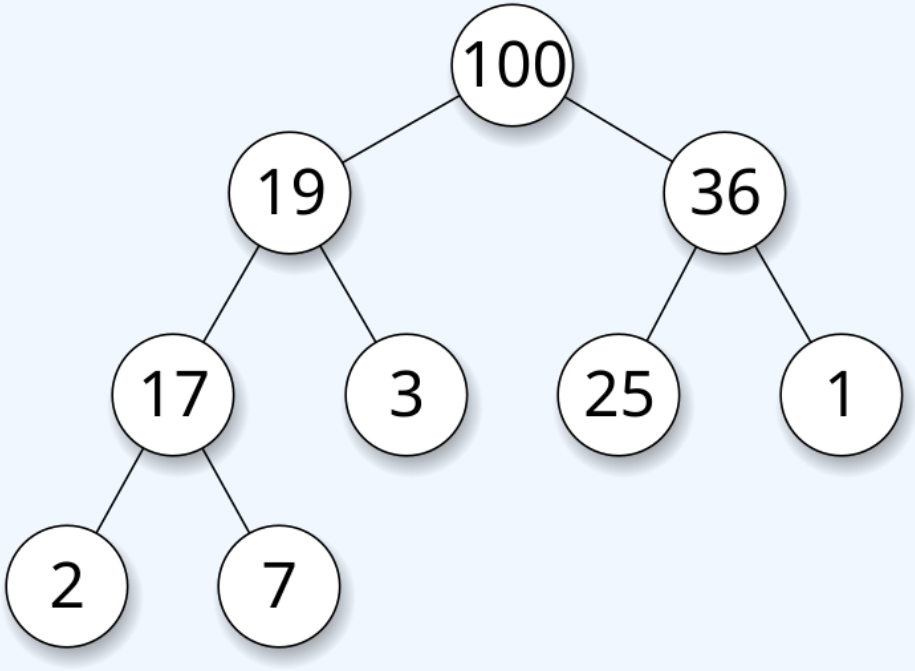
Representando um heap

Um heap é uma árvore binária completa armazenada em vetor $A[1 \ldots n]$

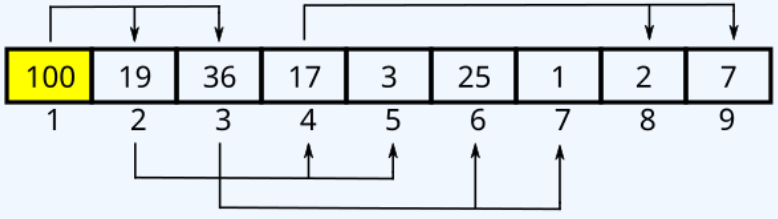
- Filhos de um nó i
  - o filho esquerdo é $2i$
  - o filho direito é $2i + 1$
- Pais
  - o pai de um nó i é $\lfloor i/2 \rfloor$
  - o nó 1 (raiz) não tem pai.
- Folhas
  - um nó i é folha se não tiver filhos, i.e., se $2i > n$.
  - as folhas são $\lfloor n/2 \rfloor + 1, \ldots, n - 1, n$.

Exemplo de um heap

Tree representation



Array representation



Propriedades básicas

- Por ser uma árvore completa, para um nó ter filho a direita, precisa ter filho a esquerda

Propriedades básicas

- Por ser uma árvore completa, para um nó ter filho a direita, precisa ter filho a esquerda
- Dado um vetor v com n elementos, como saber se o i-ésimo nó é uma folha?
  - nó i não tem filhos
  - $2i > n$? Ou seja, $i > n/2$?

Propriedades básicas

- Por ser uma árvore completa, para um nó ter filho a direita, precisa ter filho a esquerda
- Dado um vetor v com n elementos, como saber se o i-ésimo nó é uma folha?
  - nó i não tem filhos
  - $2i > n$? Ou seja, $i > n/2$?
- (Corolário) Metade dos nós são folhas

Propriedades básicas

- Por ser uma árvore completa, para um nó ter filho a direita, precisa ter filho a esquerda
- Dado um vetor v com n elementos, como saber se o i-ésimo nó é uma folha?
  - nó i não tem filhos
  - $2i > n$? Ou seja, $i > n/2$?
- (Corolário) Metade dos nós são folhas
- Quantos nós há em cada nível?

Propriedades básicas

- Por ser uma árvore completa, para um nó ter filho a direita, precisa ter filho a esquerda
- Dado um vetor v com n elementos, como saber se o i-ésimo nó é uma folha?
  - nó i não tem filhos
  - $2i > n$? Ou seja, $i > n/2$?
- (Corolário) Metade dos nós são folhas
- Quantos nós há em cada nível?
  - O primeiro (nível zero) tem apenas a raiz
  - O segundo (nível 1) tem dois filhos da raiz
  - O terceiro (nível 2) tem 4 nós
  - ...
  - Com exceção do último, o ℓ-ésimo nível tem 2^ℓ nós

Propriedades básicas

- Por ser uma árvore completa, para um nó ter filho a direita, precisa ter filho a esquerda
- Dado um vetor v com n elementos, como saber se o i-ésimo nó é uma folha?
  - nó i não tem filhos
  - $2i > n$? Ou seja, $i > n/2$?
- (Corolário) Metade dos nós são folhas
- Quantos nós há em cada nível?
  - O primeiro (nível zero) tem apenas a raiz
  - O segundo (nível 1) tem dois filhos da raiz
  - O terceiro (nível 2) tem 4 nós
  - ...
  - Com exceção do último, o ℓ-ésimo nível tem 2^ℓ nós
- Quais nós estão em cada nível?

Propriedades básicas

- Por ser uma árvore completa, para um nó ter filho a direita, precisa ter filho a esquerda
- Dado um vetor v com n elementos, como saber se o i-ésimo nó é uma folha?
  - nó i não tem filhos
  - $2i > n$? Ou seja, $i > n/2$?
- (Corolário) Metade dos nós são folhas
- Quantos nós há em cada nível?
  - O primeiro (nível zero) tem apenas a raiz
  - O segundo (nível 1) tem dois filhos da raiz
  - O terceiro (nível 2) tem 4 nós
  - ...
  - Com exceção do último, o ℓ-ésimo nível tem 2^ℓ nós
- Quais nós estão em cada nível?
  - se i está no nível ℓ, o número nós nos níveis anteriores é

$$2^0 + 2^1 + \cdots + 2^{\ell-1} = 2^\ell - 1$$

  - então os nós do nível ℓ são os de índices

$$2^\ell, 2^\ell + 1, 2^\ell + 2, \ldots, 2^{\ell+1} - 1$$

Propriedades básicas

Dado um vetor v representando um heap com n elementos e um índice i, qual o nível do nó i?

Propriedades básicas

Dado um vetor v representando um heap com n elementos e um índice i, qual o nível do nó i?

Nível de um nó

O nível de um nó de índice i é $\lfloor \log_2 i \rfloor$.

- se i está no nível ℓ, então

$$2^\ell \leq i < 2^{\ell+1} \Rightarrow$$

Propriedades básicas

Dado um vetor v representando um heap com n elementos e um índice i, qual o nível do nó i?

Nível de um nó

O nível de um nó de índice i é $\lfloor \log_2 i \rfloor$.

- se i está no nível ℓ, então

$$\begin{array}{rcccl} 2^\ell & \leq & i & < & 2^{\ell+1} & \Rightarrow \\ \log_2 2^\ell & \leq & \log_2 i & < & \log_2 2^{\ell+1} & \Rightarrow \end{array}$$

Propriedades básicas

Dado um vetor v representando um heap com n elementos e um índice i, qual o nível do nó i?

Nível de um nó

O nível de um nó de índice i é $\lfloor \log_2 i \rfloor$.

- se i está no nível ℓ, então

$$\begin{array}{rcccl} 2^\ell & \leq & i & < & 2^{\ell+1} & \Rightarrow \\ \log_2 2^\ell & \leq & \log_2 i & < & \log_2 2^{\ell+1} & \Rightarrow \\ \ell & \leq & \log_2 i & < & \ell + 1 & \end{array}$$

Propriedades básicas

Dado um vetor v representando um heap com n elementos e um índice i, qual o nível do nó i?

Nível de um nó

O nível de um nó de índice i é $\lfloor \log_2 i \rfloor$.

- se i está no nível ℓ, então

$$\begin{array}{rcccl} 2^\ell & \leq & i & < & 2^{\ell+1} \quad \Rightarrow \\ \log_2 2^\ell & \leq & \log_2 i & < & \log_2 2^{\ell+1} \quad \Rightarrow \\ \ell & \leq & \log_2 i & < & \ell + 1 \end{array}$$

- assim, $\ell = \lfloor \log_2 i \rfloor$

Propriedades básicas

Dado um vetor v representando um heap com n elementos e um índice i, qual o nível do nó i?

Nível de um nó

O nível de um nó de índice i é $\lfloor \log_2 i \rfloor$.

- se i está no nível ℓ, então

$$\begin{array}{rcccl} 2^\ell & \leq & i & < & 2^{\ell+1} \quad \Rightarrow \\ \log_2 2^\ell & \leq & \log_2 i & < & \log_2 2^{\ell+1} \quad \Rightarrow \\ \ell & \leq & \log_2 i & < & \ell + 1 \end{array}$$

- assim, $\ell = \lfloor \log_2 i \rfloor$

Corolário

A altura de um heap com n nós $\lfloor \log_2 n \rfloor$.

Definição

Um heap $v[1 \ldots n]$ é chamado de max-heap se cada nó for uma folha ou tiver valor maior que seus filhos, i.e., para $1 \leq i \leq n/2$,

- $v[i] \geq v[2i]$ e $v[i] \geq v[2i+1]$

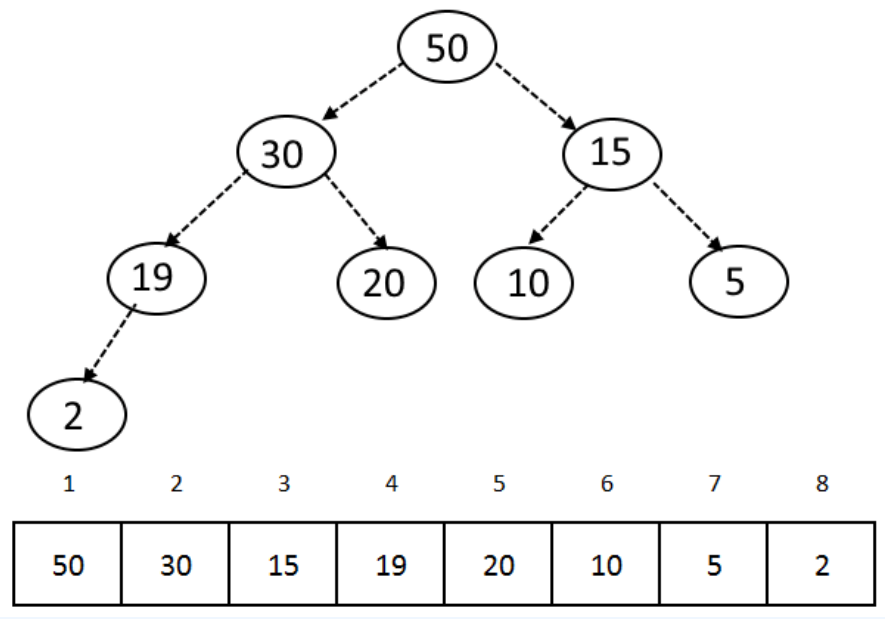
Definição

Um heap $v[1 \ldots n]$ é chamado de max-heap se cada nó for uma folha ou tiver valor maior que seus filhos, i.e., para $1 \leq i \leq n/2$,

- $v[i] \geq v[2i]$ e $v[i] \geq v[2i+1]$

- essa restrição é a propriedade de max-heap
- o valor da raiz é um máximo do heap
- cada subárvore também é um max-heap
- um max-heap não é uma árvore de busca binária!

Exemplo de max-heap



Consertando um max-heap

Problema

Temos um vetor v com n nós que é quase um max-heap, mas tem apenas um nó, digamos, k, que não respeita a propriedade de max-heap, ou seja, é menor do que algum dos filhos.
Como consertar v para ele ser um max-heap?

Consertando um max-heap

Problema

Temos um vetor v com n nós que é quase um max-heap, mas tem apenas um nó, digamos, k, que não respeita a propriedade de max-heap, ou seja, é menor do que algum dos filhos.
Como consertar v para ele ser um max-heap?

De uma forma mais geral, digamos que k é a raiz de um heap e os heaps a esquerda e a direita são max-heaps.
Como consertar v para que k passe a ser um max-heap?

MaxHeapify

Ideia: consertar a raiz, possivelmente introduzindo um problema a esquerda ou a direita, então consertar recursivamente o que ficou problemático.

MaxHeapify

Ideia: consertar a raiz, possivelmente introduzindo um problema a esquerda ou a direita, então consertar recursivamente o que ficou problemático.

`MaxHeapify`(v, n, k)

Input: $v \in \mathbb{N}^n, k \in \mathbb{N} : 1 \leq k \leq n$

$e \leftarrow 2k$
$d \leftarrow 2k + 1$
$i_{max} \leftarrow k$
if $e \leq n$ *and* $A[e] > A[i_{max}]$ **then**
  $i_{max} \leftarrow e$
if $d \leq n$ *and* $A[d] > A[i_{max}]$ **then**
  $i_{max} \leftarrow d$
if $i_{max} \neq k$ **then**
  $v[k] \leftrightarrow v[i_{max}]$
  `MaxHeapify`(v, n, i_{max})

Exemplo na lousa...

Correção de MaxHeapify

Lema

`MaxHeapify` transforma a subárvore k em max-heap.

Ideia para demonstração: indução na altura h do nó k.

- se $h = 0$, então k é folha e o algoritmo está correto
- considere um nó k com altura $h > 0$
- suponha que o algoritmo funciona para árvores menores
  - antes da linha 8, temos $v[i_{max}] \geq v[k], v[2k], v[2k+1]$
  - após a linha 9, temos $v[k] \geq v[2k], v[2k+1]$
  - segue que $v[k]$ é máximo no subvetor
  - pela hipótese de indução, `MaxHeapify` transforma a subárvore com raiz i_{max} em max-heap
  - segue que k é max-heap

Correção de MaxHeapify

Lema

`MaxHeapify` termina

Correção de `MaxHeapify`

Lema

`MaxHeapify` termina

Lema

`MaxHeapify` transforma a subárvore k em max-heap.

Correção de `MaxHeapify`

Lema

`MaxHeapify` termina

Lema

`MaxHeapify` transforma a subárvore k em max-heap.

Teorema

`MaxHeapify` está correto.

Complexidade de MaxHeapify

Se tentarmos escrever em função da quantidade de nós, obtemos, para o pior caso, algo como

$$T(n) = T(m) + \Theta(1)$$

onde m é a quantidade de nós a esquerda ou a direita...

Complexidade de MaxHeapify

Se tentarmos escrever em função da quantidade de nós, obtemos, para o pior caso, algo como

$$T(n) = T(m) + \Theta(1)$$

onde m é a quantidade de nós a esquerda ou a direita...

No entanto, como um heap é uma árvore completa, podemos pensar na altura da árvore:

$$R(h) = R(h-1) + \Theta(1)$$

Assim, obtemos $R(h) \in O(h)$.

Portanto, numa árvore com n nós, temos altura $h = \lfloor \log n \rfloor$, e a quantidade de operações básicas é

$$T(n) \in O(\log n)$$

Construindo um max-heap

Problema

Dado um vetor v com n nós, como transformamos v num max-heap?

Construindo um max-heap

Problema

Dado um vetor v com n nós, como transformamos v num max-heap?

- tentar consertar da raiz para as folhas é problemático!

Construindo um max-heap

Problema

Dado um vetor v com n nós, como transformamos v num max-heap?

- tentar consertar da raiz para as folhas é problemático!
- cada folha já é um max-heap (com um único nó)
- consertamos o penúltimo nível
- depois o antepenúltimo
- e assim por diante

Construindo um max-heap

Problema

Dado um vetor v com n nós, como transformamos v num max-heap?

- tentar consertar da raiz para as folhas é problemático!
- cada folha já é um max-heap (com um único nó)
- consertamos o penúltimo nível
- depois o antepenúltimo
- e assim por diante

`BuildMaxHeap`(v, n)

Input: $v \in \mathbb{N}^n$

1 $i \leftarrow \lfloor n/2 \rfloor$
2 **while** $i \geq 1$ **do**
3 $\quad$ `MaxHeapify`(v, n, i)
4 $\quad$ $i \leftarrow i - 1$

Construindo um max-heap

Problema

Dado um vetor v com n nós, como transformamos v num max-heap?

- tentar consertar da raiz para as folhas é problemático!
- cada folha já é um max-heap (com um único nó)
- consertamos o penúltimo nível
- depois o antepenúltimo
- e assim por diante

`BuildMaxHeap`(v, n)

Input: $v \in \mathbb{N}^n$

1 $i \leftarrow \lfloor n/2 \rfloor$
2 **while** $i \geq 1$ **do**
3 $\quad$ `MaxHeapify`(v, n, i)
4 $\quad$ $i \leftarrow i - 1$

Análise de BuildMaxHeap

BuildMaxHeap(v, n)

Input: $v \in \mathbb{N}^n$

```
i ← ⌊n/2⌋
while i ≥ 1 do
    MaxHeapify(v, n, i)
    i ← i − 1
```

Terminação

$\forall n \in \mathbb{N}^*$ BuildMaxHeap(v, n) termina

Análise de BuildMaxHeap

BuildMaxHeap(v, n)

Input: $v \in \mathbb{N}^n$

```
i ← ⌊n/2⌋
while i ≥ 1 do
    MaxHeapify(v, n, i)
    i ← i − 1
```

Terminação

$\forall n \in \mathbb{N}^*$ BuildMaxHeap(v, n) termina

Invariante

No início da i-ésima iteração, $i + 1, \ldots, n$ são raízes de max-heaps.

Análise de BuildMaxHeap

BuildMaxHeap(v, n)

Input: $v \in \mathbb{N}^n$

```
i ← ⌊n/2⌋
while i ≥ 1 do
    MaxHeapify(v, n, i)
    i ← i − 1
```

Terminação

$\forall n \in \mathbb{N}^*$ BuildMaxHeap(v, n) termina

Invariante

No início da i-ésima iteração, $i + 1, \ldots, n$ são raízes de max-heaps.

Teorema

$\forall n \in \mathbb{N}^*$ BuildMaxHeap(v, n) está correto

Perguntas ou comentários?